

PREFACE FOR FACULTY

The Cooley-Tukey Radix-2 Decimation-in-Time (DIT) Fast Fourier Transform is one of the most fundamental algorithms in modern computing. It exploits the divide-and-conquer strategy by recursively decomposing an N -point DFT into smaller DFTs of even and odd index sequences.

Pedagogical Strategy:

1. Mathematically derive the decomposition of an N -point DFT into two $N/2$ -point DFTs:
$$X[k] = X_e[k] + W_N^k X_o[k].$$
2. Demonstrate how twiddle factor periodicity and symmetry ($W_N^{k+N/2} = -W_N^k$) allow computing two outputs from one butterfly.
3. Draw the step-by-step Signal Flow Graph (SFG) for $N = 8$: Show 3 stages ($M = \log_2 8 = 3$), with 4 butterflies per stage.
4. Explain **Bit-Reversal Indexing**: Show the binary reversal mechanism for arranging the inputs.
5. Emphasize **In-Place Computation**: How dual outputs overwrite inputs in memory registers, eliminating external memory requirements.

1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Derive** the Radix-2 DIT-FFT butterfly equations from the standard DFT definition.
2. **Construct** complete 4-point and 8-point DIT-FFT signal flow graphs with correct twiddle factors.
3. **Compute** bit-reversed index mappings for arbitrary powers of 2.
4. **Evaluate** intermediate and final butterfly outputs for given numerical sequences.
5. **Analyze** memory access and computational complexity ($O(N \log_2 N)$).

2. MATHEMATICAL FOUNDATIONS

2.1 Derivation of the Radix-2 DIT Algorithm

Let $x[n]$ be an N -point sequence where $N = 2^M$. Split $x[n]$ into even-indexed and odd-indexed samples:

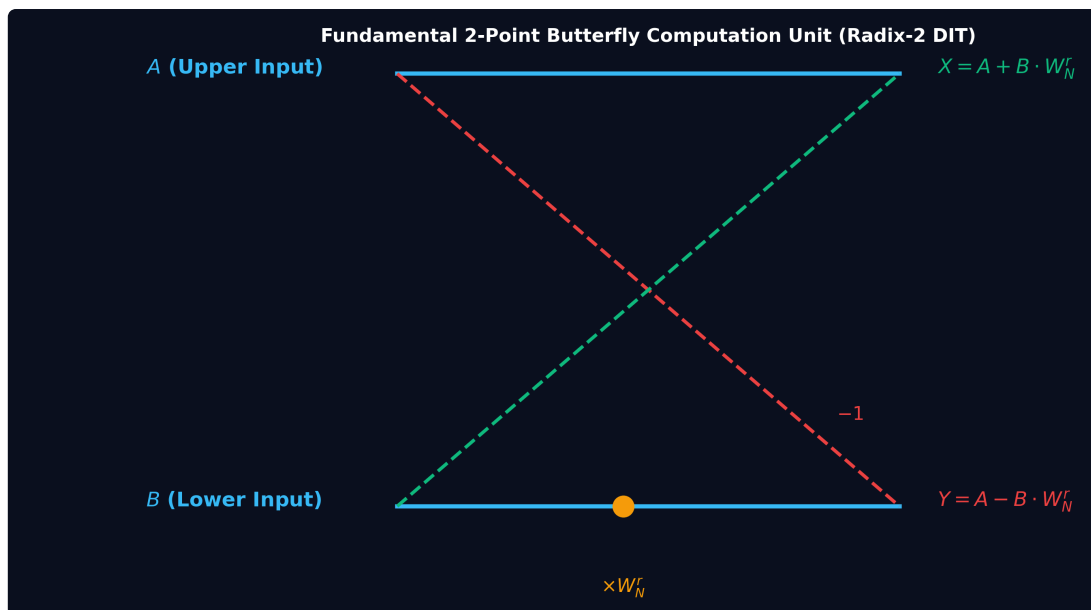
$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk} = \sum_{r=0}^{N/2-1} x[2r] W_N^{2rk} + \sum_{r=0}^{N/2-1} x[2r+1] W_N^{(2r+1)k}$$

Recall that $W_N^2 = e^{-j\frac{2\pi}{N} \cdot 2} = e^{-j\frac{2\pi}{N/2}} = W_{N/2}$:

$$X[k] = \sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{rk} + W_N^k \sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{rk}$$

Let $X_e[k] = \text{DFT}_{N/2}\{x[2r]\}$ and $X_o[k] = \text{DFT}_{N/2}\{x[2r+1]\}$. Then $X[k] = X_e[k] + W_N^k X_o[k]$.
Because $X_e[k]$ and $X_o[k]$ are periodic with period $N/2$: $X_e[k + N/2] = X_e[k]$, $X_o[k + N/2] = X_o[k]$.
Using the twiddle symmetry $W_N^{k+N/2} = -W_N^k$: $X[k] = X_e[k] + W_N^k X_o[k]$, $0 \leq k < N/2$
 $X[k + N/2] = X_e[k] - W_N^k X_o[k]$, $0 \leq k < N/2$

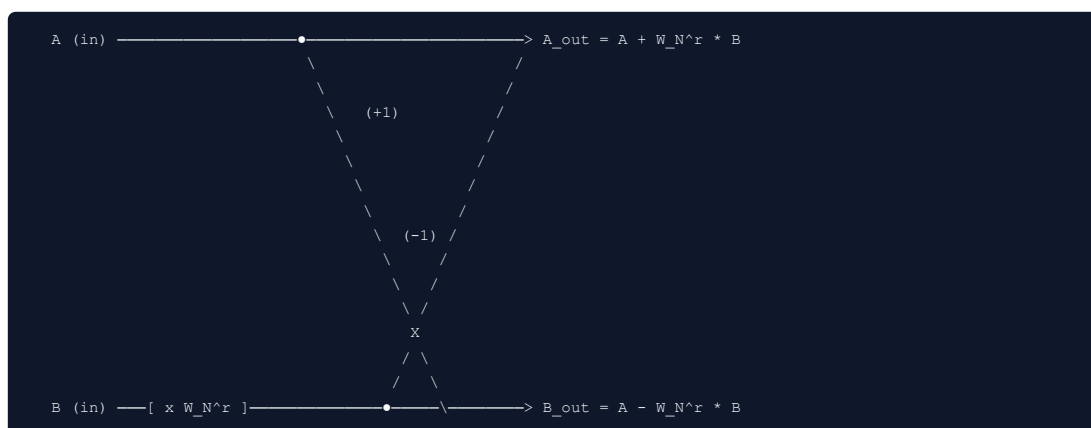
2.2 The DIT Butterfly Processing Element



The basic 2-point butterfly computation takes inputs A and B , multiplies B by twiddle factor W_N^r , and computes:

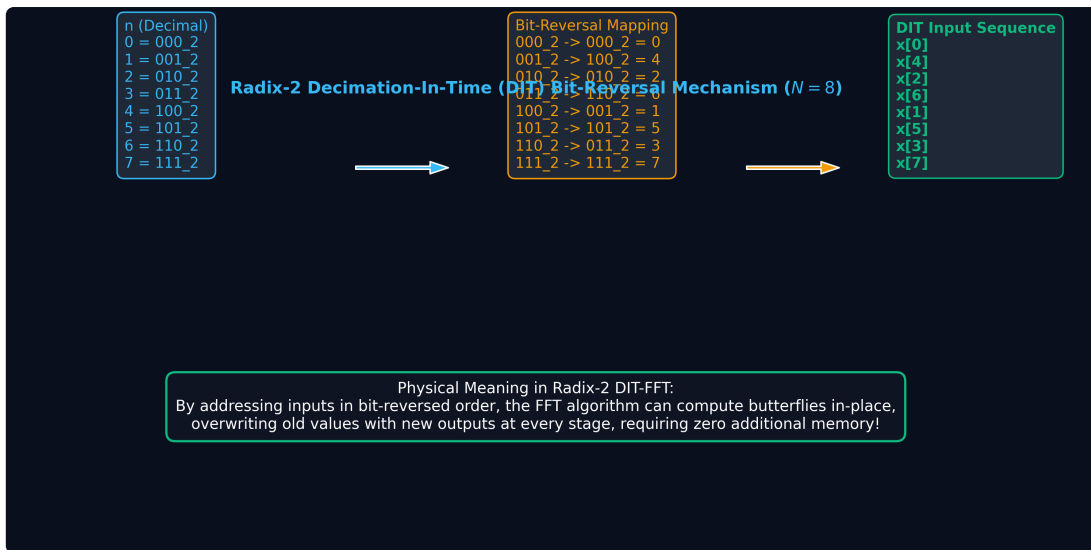
$$A_{\text{out}} = A + W_N^r B$$

$$B_{\text{out}} = A - W_N^r B$$



- **Multiplications per butterfly:** 1 complex multiplication.
- **Additions per butterfly:** 2 complex additions.
- **Butterflies per stage:** $N/2$.
- **Total stages:** $M = \log_2 N$.
- **Total Complex Multiplications:** $\frac{N}{2} \log_2 N$.
- **Total Complex Additions:** $N \log_2 N$.

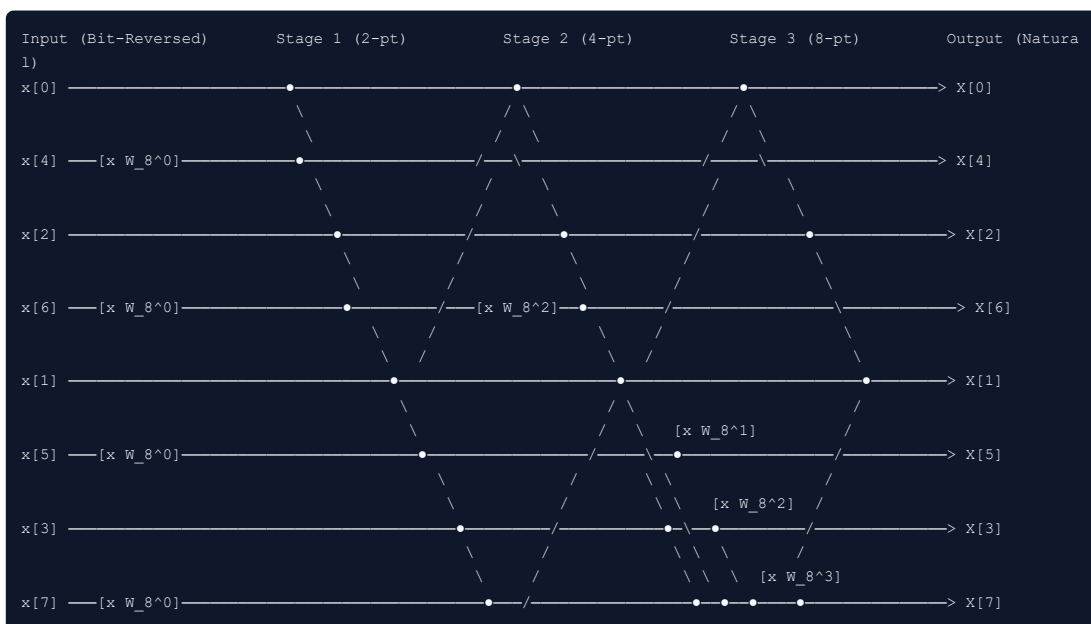
2.3 Bit-Reversal Permutation & Decimation Tree



For an $N = 8$ DIT-FFT, the input array is ordered by reversing the 3-bit binary representation of index n :

Natural Index n	Binary ($b_2b_1b_0$)	Reversed Binary ($b_0b_1b_2$)	Bit-Reversed Index n_{rev}	Input Assigned
0	000	000	0	$x[0]$
1	001	100	4	$x[4]$
2	010	010	2	$x[2]$
3	011	110	6	$x[6]$
4	100	001	1	$x[1]$
5	101	101	5	$x[5]$
6	110	011	3	$x[3]$
7	111	111	7	$x[7]$

2.4 Complete 8-Point DIT-FFT Signal Flow Graph (SFG)



3. WORKED NUMERICAL EXAMPLES

Example 10.1: 8-Point DIT-FFT Butterfly Execution

Problem: Compute the 8-point DFT of $x[n] = 1, 2, 3, 4, 4, 3, 2, 1$ using the Radix-2 DIT-FFT algorithm.

Solution:

1. **Input Bit-Reversal:** $x_{\text{rev}} = [x[0], x[4], x[2], x[6], x[1], x[5], x[3], x[7]] = [1, 4, 3, 2, 2, 3, 4, 1]$.
2. **Stage 1 (2-Point Butterflies, Twiddle $W_8^0 = 1$):**
 - Pair (0, 1): $v_1[0] = 1 + 4 = 5$; $v_1[1] = 1 - 4 = -3$
 - Pair (2, 3): $v_1[2] = 3 + 2 = 5$; $v_1[3] = 3 - 2 = 1$
 - Pair (4, 5): $v_1[4] = 2 + 3 = 5$; $v_1[5] = 2 - 3 = -1$
 - Pair (6, 7): $v_1[6] = 4 + 1 = 5$; $v_1[7] = 4 - 1 = 3$ Stage 1 output: $v_1 = [5, -3, 5, 1, 5, -1, 5, 3]$.
3. **Stage 2 (4-Point Butterflies, Twiddles $W_8^0 = 1, W_8^2 = -j$):**
 - Group 1:
 - $v_2[0] = v_1[0] + W_8^0 v_1[2] = 5 + 5 = 10$
 - $v_2[1] = v_1[1] + W_8^2 v_1[3] = -3 + (-j)(1) = -3 - j$
 - $v_2[2] = v_1[0] - W_8^0 v_1[2] = 5 - 5 = 0$
 - $v_2[3] = v_1[1] - W_8^2 v_1[3] = -3 - (-j)(1) = -3 + j$
 - Group 2:
 - $v_2[4] = v_1[4] + W_8^0 v_1[6] = 5 + 5 = 10$
 - $v_2[5] = v_1[5] + W_8^2 v_1[7] = -1 + (-j)(3) = -1 - 3j$
 - $v_2[6] = v_1[4] - W_8^0 v_1[6] = 5 - 5 = 0$
 - $v_2[7] = v_1[5] - W_8^2 v_1[7] = -1 - (-j)(3) = -1 + 3j$
4. **Stage 3 (8-Point Butterflies, Twiddles $W_8^0 = 1, W_8^1 = \frac{\sqrt{2}}{2}(1 - j), W_8^2 = -j, W_8^3 = -\frac{\sqrt{2}}{2}(1 + j)$):**
 - $X[0] = v_2[0] + W_8^0 v_2[4] = 10 + 10 = 20$
 - $X[1] = v_2[1] + W_8^1 v_2[5] = (-3 - j) + (0.7071 - 0.7071j)(-1 - 3j) = -3 - j - 2.8284 - 1.4142j = -5.8284 - 2.4142j$
 - $X[2] = v_2[2] + W_8^2 v_2[6] = 0 + (-j)(0) = 0$
 - $X[3] = v_2[3] + W_8^3 v_2[7] = (-3 + j) + (-0.7071 - 0.7071j)(-1 + 3j) = -0.1716 - 0.4142j$
 - $X[4] = v_2[0] - W_8^0 v_2[4] = 10 - 10 = 0$
 - $X[5] = X[3]^* = -0.1716 + 0.4142j$
 - $X[6] = X[2]^* = 0$
 - $X[7] = X[1]^* = -5.8284 + 2.4142j$

4. UNIVERSITY EXAMINATION QUESTIONS & MARKING RUBRIC

Question 1 (15 Marks)

(a) Derive the Radix-2 Decimation-in-Time FFT algorithm for $N = 8$. Draw the complete signal flow graph showing all butterfly connections and twiddle factor exponents. (10 Marks) (b) What is bit reversal? Explain the algorithm to bit-reverse an array of length $N = 16$. (5 Marks)

Model Answer & Step-by-Step Marking Rubric:

- **Part (a):**
 - Mathematical derivation splitting into even and odd parts (3 Marks)
 - Butterfly equations $A \pm W_N^r B$ (2 Marks)
 - Neatly drawn 8-point SFG with 3 stages, labeled nodes, branch multipliers (-1) , and twiddle factors $W_8^0, W_8^1, W_8^2, W_8^3$ (5 Marks)
- **Part (b):**
 - Definition of bit reversal and indexing table for $N = 16$ ($b_3 b_2 b_1 b_0 \leftrightarrow b_0 b_1 b_2 b_3$) (3 Marks)
 - In-place swapping code logic / explanation (2 Marks)

5. PYTHON VERIFICATION SCRIPT

```
import numpy as np

x = np.array([1, 2, 3, 4, 4, 3, 2, 1])
X = np.fft.fft(x)
print("8-Point DIT-FFT Output:")
for k, val in enumerate(X):
    print(f"X[{k}] = {val.real:.4f} + {val.imag:.4f}j")
```